ROBOTICS · LESSON 8

Make It Visible

Stream the robot's live state over WiFi to a browser — watch the loop chase its target and the guard act, untethered. You can't tune what you can't see.

ROBO-008 · ~50 MIN · INTERMEDIATE

1. The one idea

The robot tells you what it is doing. Stream its live state — target speed versus measured, distance travelled, whether the guard is clamping — over WiFi to a browser, and watch the control loop and the guard act in real time. You can't tune what you can't see.

2. Why it matters

Every lesson so far you debugged down a USB cable — but a robot that drives away can't stay tethered. And tuning a PID, or trusting a guard, means *seeing* it: the target line, the measured line chasing it, the moment the guard clamps. The ESP32's superpower is WiFi, so the robot can become its own little telemetry server. This is observability — the thing every roboticist does to tune and trust a loop — and it is the Monogate first rule made literal: **make the control decision visible.**

3. Parts & wiring

No new hardware — the Lesson 6/7 rover plus the ESP32's built-in WiFi, and a phone or laptop on the **same 2.4 GHz network**. The robot serves a tiny page and a live data feed; the browser draws it.

4. Predict (do this before uploading)

- When the guard clamps (an obstacle comes inside the safe distance), what will the telemetry show — and how fast?
- When you pinch a wheel so the PID fights, what will the *target* and *measured* lines do relative to each other?
- The page polls ten times a second. Where does that data come from if your control loop is busy driving?

5. Build it — the robot as a tiny server

The control loop writes its state into a few globals; a web handler hands them out as JSON whenever the browser asks.

```
 #include <WiFi.h>
#include <WebServer.h>

WebServer server(80);

// Updated by your Lesson 5/6/7 control code each loop:
volatile float gTarget = 0, gMeasured = 0, gDistanceCm = 0;
volatile bool  gGuard = false;

void handleData() {
  char json[160];
  snprintf(json, sizeof(json),
    "{\"target\":%.0f,\"measured\":%.0f,\"distance\":%.1f,\"guard\":%s}",
    gTarget, gMeasured, gDistanceCm, gGuard ? "true" : "false");
  server.send(200, "application/json", json);
}

void setup() {
  Serial.begin(115200);
  WiFi.begin("YOUR_SSID", "YOUR_PASS");
  while (WiFi.status() ≠ WL_CONNECTED) delay(200);
  Serial.println(WiFi.localIP());      // open this address in a browser
  server.on("/data", handleData);
  server.begin();
}

void loop() {
  server.handleClient();
  //  ...  your Lesson 5/6/7 control code runs here, updating gTarget, gMeasured,  ...
}
```

A tiny page polls that feed and draws it — no framework needed:

```
<canvas id="c" width="320" height="120"></canvas>
<b id="g"></b>
<script>
setInterval(async () => {
  const s = await (await fetch('/data')).json();
  document.getElementById('g').textContent = s.guard ? 'GUARD: CLAMPING' : 'guard: clear';
  // … push s.target / s.measured into a scrolling line on the canvas …
}, 100);
</script>
```

The shape: the loop *publishes*, the browser *subscribes*. Keep the JSON tiny and the page small so serving it never stalls the control loop — which is exactly why Lesson 6 used a non-blocking timer instead of `delay()`.

6. Test against your prediction

Open the ESP32's IP in your browser and drive the rover. Watch `measured` chase `target`, and watch the **GUARD** indicator light the instant your hand comes inside the safe distance — the guard's decision, visible and untethered. Pinch a wheel and see the gap between the two lines that the PID is working to close.

7. What goes wrong (pre-loaded debugging)

- **Never connects** → wrong SSID/password, or a **5 GHz** network — the ESP32 is 2.4 GHz only. The classic WiFi trap.
- **Page loads but no numbers** → the control loop is blocking `server.handleClient()` with a long `delay()`. Keep the loop non-blocking.
- **Robot stutters when the browser connects** → you're serving a heavy page from the loop. Keep the page tiny and the JSON small.
- **Numbers are frozen** → you forgot to update the `g…` globals from the control code. The feed is only as live as what writes to it.
- **Plot lags or tears** → polling too fast or doing heavy work per frame; 10 Hz of small JSON is plenty.

8. Checkpoint

The robot reports its own state — live, untethered, from a browser — while it drives. You can see the loop chase its target and the guard act. That's observability: the Monogate first rule, *make the control decision visible*, running on your own robot. You can't tune or trust what you can't see; now you can see it. And with that, the track is complete — actuate, guard, sense, locomote, close the loop, control it, place it, and watch it.

9. Push further

- **Teleop:** add buttons that POST drive commands back, and steer the rover from your phone.
- **A replayable trace:** log the JSON frames to a file — the Monogate evidence idea, on a robot.
- **Mark the clamps:** plot every guard event as a tick on the timeline — a visible safety record you can scroll back through.
